

NAME:- BHAVANI SANKAR

REG.NO:- 192411238

SESSION 2 LAB EXPERIMENTS

1. Write a program that takes an input list of n numbers and creates a new list containing

only the unique elements from the original list. What is the space complexity of the

algorithm?

Code:-

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, a[100], b[100], i, j, k = 0, found;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    for (i = 0; i < n; i++) {
```

```
        found = 0;
```

```
        for (j = 0; j < k; j++) {
```

```
            if (a[i] == b[j]) {
```

```
        found = 1;
        break;
    }
}

if (!found)
    b[k++] = a[i];
}

printf("List after removing duplicates: ");
for (i = 0; i < k; i++)
    printf("%d ", b[i]);

return 0;
}
```

OUTPUT:-

```
Output

Enter number of elements: 5
Enter elements:
10
10
52
45
52
List after removing duplicates: 10 52 45
```

2. Sort an array of integers using the bubble sort technique. Analyze its time complexity

using Big-O notation. Write the code

CODE:-

```
#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
```

```
    printf("%d ", a[i]);

    return 0;
}
```

OUTPUT:-

```
Output

Enter number of elements: 5
Enter elements:
58
12
94
12
10
Sorted array: 10 12 12 58 94
```

3. Checks if a given number x exists in a sorted array arr using binary search.

Analyze its

time complexity using Big-O notation.

CODE:-

```
#include <stdio.h>
```

```
int main() {
    int a[100], n, i, x;
    int low, high, mid, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```
printf("Enter sorted elements in ascending order:\n");
```

```
for (i = 0; i < n; i++)
```

```
    scanf("%d", &a[i]);
```

```
printf("Enter number to search: ");
```

```
scanf("%d", &x);
```

```
low = 0;
```

```
high = n - 1;
```

```
while (low <= high) {
```

```
    mid = low + (high - low) / 2;
```

```
    if (a[mid] == x) {
```

```
        found = 1;
```

```
        break;
```

```
    } else if (a[mid] < x) {
```

```
        low = mid + 1;
```

```
    } else {
```

```
        high = mid - 1;
```

```
    }
```

```
}
```

```
if (found)
```

```
    printf("Element found");
```

```
else
```

```
printf("Element not found");

return 0;
}
```

OUTPUT:-

```
Output

Enter number of elements: 5
Enter sorted elements in ascending order:
10
11
15
18
19
Enter number to search: 15
Element found
```

4. Given an array of integers nums, sort the array in ascending order and return it. You

must solve the problem without using any built-in functions in $O(n \log(n))$ time complexity

and with the smallest space complexity possible.

CODE:-

```
#include <stdio.h>
```

```
void merge(int a[], int temp[], int left,
           int mid, int right) {
    int i = left, j = mid + 1, k = left;
```

```
while (i <= mid && j <= right) {  
    if (a[i] <= a[j])  
        temp[k++] = a[i++];  
    else  
        temp[k++] = a[j++];  
}
```

```
while (i <= mid)  
    temp[k++] = a[i++];
```

```
while (j <= right)  
    temp[k++] = a[j++];
```

```
for (i = left; i <= right; i++)  
    a[i] = temp[i];  
}
```

```
void mergeSort(int a[], int temp[], int left, int right) {  
    if (left < right) {  
        int mid = left + (right - left) / 2;  
  
        mergeSort(a, temp, left, mid);  
        mergeSort(a, temp, mid + 1, right);  
  
        merge(a, temp, left, mid, right);  
    }  
}
```

```
int main() {  
    int a[100], temp[100], n, i;  
  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
  
    printf("Enter elements:\n");  
    for (i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
  
    mergeSort(a, temp, 0, n - 1);  
  
    printf("Sorted array: ");  
    for (i = 0; i < n; i++)  
        printf("%d ", a[i]);  
  
    return 0;  
}
```

OUTPUT:-

Output

Enter number of elements: 5

Enter elements:

1

2

8

9

94

Sorted array: 1 2 8 9 94